

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ УКРАЇНИ
“КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ
імені ІГОРЯ СІКОРСЬКОГО”

Факультет програмних систем та прикладної математики
Кафедра програмного забезпечення комп’ютерних систем



Комп’ютерний практикум №2
з дисципліни **“Програмне забезпечення технології цифрових двійників”**
тема **“Розроблення IoT-каналу даних цифрового двійника”**

Виконав
студент V курсу
групи КП-61мн
Семенюк Сергій Іванович

Київ 2026

Тема: розроблення програмних компонентів IoT-каналу даних: емуляція сенсорів та передавання телеметрії за протоколом MQTT.

Мета: набути практичних умінь інтеграції сенсорних даних (IoT) у програмні системи та застосування протоколів передавання даних інтернету речей.

Завдання: розробити емулятор сенсорів фізичного об'єкта та програмні компоненти передавання і приймання телеметрії через MQTT-брокер.

Кроки реалізації:

1. Визначити перелік сенсорів об'єкта (не менше 4–6 вимірюваних величин) та параметри телеметрії: діапазони значень, частоту вимірювань, одиниці вимірювання.
2. Розробити емулятор сенсорів (Python, Node.js або інша мова за вибором), який генерує реалістичні часові ряди значень із урахуванням трендів, добових циклів, випадкового шуму та періодичних аномалій.
3. Розгорнути MQTT-брокер (наприклад, Eclipse Mosquitto — локально або у Docker-контейнері).
4. Спроєктувати структуру MQTT-топіків (наприклад, object/{id}/sensor/{name}) та формат повідомлень телеметрії у JSON (значення, часова мітка, ідентифікатор сенсора, одиниці вимірювання).
5. Реалізувати компонент-видавець (publisher), який передає телеметрію від емулятора сенсорів до брокера із заданою частотою.
6. Реалізувати компонент-підписник (subscriber), який приймає повідомлення, виконує валідацію їх структури та веде журнал отриманих даних.
7. Дослідити вплив рівнів QoS (0, 1, 2) на доставку повідомлень; налаштувати механізм Last Will для виявлення від'єднання джерела даних.
8. Перевірити працездатність каналу: одночасна робота кількох емульованих сенсорів, оцінка втрат і затримок повідомлень; оформити звіт.

Вимоги до змісту звіту: перелік сенсорів та характеристики телеметрії; схема IoT-каналу даних; структура топіків і формат повідомлень; опис реалізації емулятора, видавця та підписника з фрагментами коду; результати дослідження рівнів QoS; знімки екрана роботи системи; висновки.

Захист роботи: демонстрація передавання телеметрії від емульованих сенсорів через брокер у реальному часі. Орієнтовні контрольні запитання: архітектура протоколу MQTT та порівняння з OPC UA і CoAP;

призначення рівнів QoS; периферійні (edge), туманні (fog) та хмарні обчислення в архітектурі цифрових двійників.

Хід роботи

1. Визначити перелік сенсорів об’єкта (не менше 4–6 вимірюваних величин) та параметри телеметрії: діапазони значень, частоту вимірювань, одиниці вимірювання.

Перш ніж перейти до програмної емуляції, важливо чітко розмежувати дві пов'язані сутності:

- **Сенсори об’єкта (апаратний рівень):** фізичні датчики, пристрої та бортові контролери, встановлені на вузлах електроскутера (наприклад, датчики Холла, термістори, шунтові датчики струму), які безпосередньо вимірюють фізичні величини у реальному середовищі.
- **Параметри телеметрії (інформаційний рівень):** оцифровані змінні, що надходять від сенсорів до програмної системи цифрового двійника. Вони містять не лише чисельне значення, а й унікальний ідентифікатор, одиниці вимірювання, частоту опитування ($f_д$), діапазон та часову мітку (Timestamp).

Спираючись на специфікацію об'єкта дослідження з попереднього комп’ютерного практикуму, згадаймо базовий перелік вимірюваних фізичних величин:

№	Телеметричний сигнал	Тип	Компонент	Частота опитування
1	Швидкість руху	Фізична величина	Ходова частина	10 Гц
2	Частота обертання електродвигуна	Фізична величина	Електродвигун	10 Гц
3	Струм акумуляторної батареї	Фізична величина	Акумуляторна батарея	1 Гц
4	Напруга акумуляторної батареї	Фізична величина	Акумуляторна батарея	1 Гц
5	Температура електродвигуна	Фізична величина	Електродвигун	0,1 Гц

6	Рівень заряду акумуляторної батареї	Фізична величина	Акумуляторна батарея	0,1 Гц
7	Стан гальмування	Дискретний сигнал	Гальмівна система	Подієво
8	Стан протиугінного блокування	Дискретний сигнал	Система захисту	Подієво

Для виконання поточного комп’ютерного практикуму доповнимо ці дані параметрами телеметрії, апаратними джерелами (сенсорами), діапазонами значень та одиницями вимірювання, необхідними для проєктування IoT-каналу:

№	Телеметричний сигнал	Сенсор (Апаратне джерело)	Одиниці вимірювання	Діапазони значень	Частота
1	Швидкість руху	Датчик швидкості	км/год	0 – 60	10 Гц
2	Частота обертання електродвигуна	Датчик Холла ротора двигуна	об/хв	0 – 3000	10 Гц
3	Струм акумуляторної батареї	Шунтовий датчик струму (BMS)	А	0 – 50	1 Гц
4	Напруга акумуляторної батареї	Вольтметр / Модуль контролю батареї	В	36.0 – 54.6	1 Гц
5	Температура електродвигуна	Термістор двигуна	°C	-20 – 120	0,1 Гц
6	Рівень заряду акумуляторної батареї	Контролер заряда батареї (BMS)	%	0 – 100	0,1 Гц
7	Стан гальмування	Кінцевий вимикач гальмівної важелі	булевий (0/1)	0 або 1	Подієво

8	Стан протиугінного блокування	Модуль протиугінної сигналізації	булевий (0/1)	0 або 1	Подієво



Рис. 1.1 Схема IoT-каналу даних

Схему було зроблено з допомогою сайту https://mermaid.live.

2. Розробити емулятор сенсорів (Python, Node.js або інша мова за вибором), який генерує реалістичні часові ряди значень із урахуванням трендів, добових циклів, випадкового шуму та періодичних аномалій.

Для виконання завдання розроблено програмний емулятор сенсорів електроскутера мовою Python. Код реалізації збережено у файлі **emulator.py** (https://drive.google.com/file/d/1Ya914dShBPrOUUi-nMoN8UcLP6aInN0H/view?usp=sharing), який додається до звіту. Емулятор формує телеметричні дані шести фізичних величин і двох дискретних сигналів із різними частотами оновлення.

Генерація часових рядів із урахуванням трендів. У класі ScooterSensorEmulator реалізовано моделювання станів поїздки: простій (idle), розгін (accelerating), рух зі сталою швидкістю (cruise) і гальмування (braking). Метод update_trip_state() відповідає за перемикання між цими станами, а update_physics() розраховує зміну швидкості, струму, напруги, температури двигуна та рівня заряду батареї. Це дає змогу формувати часові ряди, значення яких залежать від поточного стану електроскутера та попередніх значень параметрів.

Урахування добових циклів. Метод ambient_temperature() моделює добові зміни температури навколишнього середовища за допомогою косинусоїдальної функції:

```
return 18.0 + 8.0 * math.cos(
    2 * math.pi * (hour - 14.0) / 24.0
)
```

У формулі 18.0 задає середню температуру середовища (°C), а 8.0 визначає амплітуду її добових коливань. Параметр hour задає поточну годину доби, а константа 14.0 визначає час досягнення максимальної температури (14:00). Період коливань становить 24 години.

Отримане значення використовується під час розрахунку цільової температури двигуна. Таким чином, модель ураховує вплив температури середовища на тепловий стан електроскутера.

Додавання випадкового шуму. Для імітації коливань вимірюваних величин використовується функція random.uniform(). Випадкові відхилення додаються до швидкості, частоти обертання двигуна, струму та температури. Це дає змогу відтворити невеликі коливання показників, характерні для сенсорних вимірювань.

Моделювання періодичних аномалій. Періодичність аномалій задається константами ANOMALY_PERIOD і ANOMALY_DURATION. Відповідно до часу роботи емулятора активується один із трьох сценаріїв: перегрів двигуна (overheat), надмірний струм (overcurrent) або падіння напруги батареї (voltage_drop). Аномалії повторюються через заданий інтервал часу та тривають протягом визначеної кількості секунд. Вибір типу аномалії реалізовано в методі update_physics():

```
phase = self.elapsed % self.ANOMALY_PERIOD

if phase < self.ANOMALY_DURATION:
    anomaly_number = int(
        self.elapsed // self.ANOMALY_PERIOD
    ) % 3
    self.anomaly_type = [
        "overheat",
        "overcurrent",
        "voltage_drop",
    ][anomaly_number]
else:
    self.anomaly_type = None
```

Змінна phase визначає, скільки часу минуло від початку поточного циклу. Якщо цей час менший за ANOMALY_DURATION, активується одна з трьох аномалій. Їх тип послідовно змінюється з кожним новим циклом. Після завершення заданої тривалості аномалія вимикається, і anomaly_type отримує значення None.

Особливість реалізації: оскільки на початку роботи емулятора self.elapsed має невелике додатне значення, перша аномалія активується практично одразу після запуску. Це зумовлено поточною умовою перевірки. Щоб відкласти появу першої аномалії на 60 секунд, необхідно замінити рядок

```
if phase < self.ANOMALY_DURATION:
```

на

```
if (
    self.elapsed >= self.ANOMALY_PERIOD
    and phase < self.ANOMALY_DURATION
):
```

У такому разі перші 60 секунд емулятор працюватиме без штучно заданих аномалій, а перша аномалія (overheat) активується після завершення першого 60-секундного інтервалу. Надалі аномалії триватимуть по 8 секунд і повторюватимуться з періодом 60 секунд. Ця зміна є необов'язковою, оскільки поточна реалізація також дає змогу продемонструвати формування телеметричних повідомлень про відхилення.

Це дає змогу перевіряти роботу системи цифрового двійника за нормальних умов і під час відхилень від штатного режиму.

Крім основних вимог кроку 2 увагу приділено:

1) **Формування телеметрії з різними частотами оновлення.**
   У методі generate_group() передбачено формування трьох груп телеметричних повідомлень: із частотою 10 Гц для швидкості та обертів двигуна, 1 Гц для струму й напруги, 0,1 Гц для температури двигуна та рівня заряду батареї. У функції main() частота формування повідомлень регулюється лічильником тактів tick_counter. Базовий інтервал роботи емулятора становить 0,1 с.
2) **Формування подієвих повідомлень.**
   Метод generate_event_messages() відстежує зміни стану гальмування та протиугінного блокування. Значення параметра brake автоматично визначається поточним станом поїздки: під час гальмування воно дорівнює True, а в інших станах – False. Параметр anti_theft набуває значення True, якщо електроскутер перебуває в режимі простою протягом щонайменше 3 секунд, і False в інших випадках.
   Метод порівнює поточні значення цих параметрів із попередніми та формує повідомлення лише за умови їх зміни. Це дає змогу передавати події активації та деактивації відповідних станів без повторного надсилання незмінених значень.
3) **Формат телеметричних повідомлень.**
   Метод make_message() формує структуру повідомлення, що містить ідентифікатор сенсора (sensor_id), назву параметра (name), його значення (value), одиницю вимірювання (unit) і часову мітку (timestamp). У функції main() повідомлення перетворюються на JSON-рядки за допомогою json.dumps() і виводяться в консоль. Це забезпечує підготовку даних до подальшого передавання через MQTT.

Отже, реалізований емулятор забезпечує генерацію телеметричних даних із різними частотами оновлення, випадковими відхиленнями, добовим температурним циклом і періодичними аномаліями. Сформовані повідомлення мають визначену структуру та можуть бути використані на наступних етапах для передавання через MQTT-брокер.

Запуск емулятора. Для запуску програми необхідно відкрити термінал, перейти до каталогу з файлом emulator.py та виконати команду запуску. У цьому випадку використовується такий каталог:

D:

cd "D:\Пари\Курс 5\семестр 1\двійники"

python emulator.py

Після запуску емулятор виводить у консоль телеметричні повідомлення у форматі JSON. Для завершення роботи програми потрібно натиснути Ctrl+C.



Рис.2.1 Результат запуску емулятора телеметрії електроскутера з аномалією перегріву двигуна протягом перших 8 секунд.

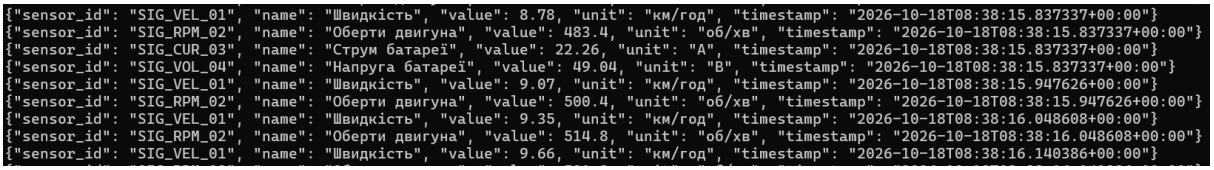


Рис.2.2 Результат роботи емулятора телеметрії електроскутера без активних аномалій.

Примітка. Активація першої аномалії одразу після запуску емулятора зумовлена умовою в коді, яка перевіряє, чи перебуває поточний момент моделювання в межах перших 8 секунд 60-секундного циклу.

3. Розгорнути MQTT-брокер (наприклад, Eclipse Mosquitto — локально або у Docker-контейнері).

Для організації обміну телеметричними даними було обрано MQTT-брокер Eclipse Mosquitto.

Спочатку з офіційного сайту Eclipse Mosquitto було завантажено інсталяційний файл для операційної системи Windows. Після цього програмне забезпечення було встановлено на комп’ютер у каталог D:\Programs\Mosquitto.

Для запуску брокера було відкрито PowerShell і виконано такі команди:

cd "D:\Programs\Mosquitto"

.\mosquitto.exe -v

```
PS C:\Users\acer> cd "D:\Programs\Mosquitto"
PS D:\Programs\Mosquitto> .\mosquitto.exe -v
1792318856: mosquitto version 2.1.2 starting
1792318856: Using default config.
1792318856: Bridge support available.
1792318856: Persistence support available.
1792318856: TLS support available.
1792318856: TLS-PSK support available.
1792318856: Websockets support available.
1792318856: Starting in local only mode. Connections will only be possible from clients running on this machine.
1792318856: Create a configuration file which defines a listener to allow remote access.
1792318856: For more details see https://mosquitto.org/documentation/authentication-methods/
1792318856: Opening ipv4 listen socket on port 1883.
1792318856: Opening ipv6 listen socket on port 1883.
1792318856: Opening http api listen socket on port 9883.
1792318856: Using http_dir C:\Program Files\Mosquitto\dashboard
1792318856: mosquitto version 2.1.2 running
1792319220: New connection from ::1:59718 on port 1883.
1792319220: New client connected from ::1:59718 as auto-94C21B21-7A92-7E47-F612-3F58803C1820 (p4, c1, k60).
1792319220: No will message specified.
1792319220: Sending CONNACK to auto-94C21B21-7A92-7E47-F612-3F58803C1820 (0, 0)
1792319220: Received SUBSCRIBE from auto-94C21B21-7A92-7E47-F612-3F58803C1820
1792319220:     test/topic (QoS 0)
1792319220: auto-94C21B21-7A92-7E47-F612-3F58803C1820 0 test/topic
1792319220: Sending SUBACK to auto-94C21B21-7A92-7E47-F612-3F58803C1820
1792319279: Received PINGREQ from auto-94C21B21-7A92-7E47-F612-3F58803C1820
1792319279: Sending PINGRESP to auto-94C21B21-7A92-7E47-F612-3F58803C1820
```

Рис.3.1 Результат виконання команд

Параметр -v забезпечує виведення докладних повідомлень про роботу брокера.

Для перевірки обміну MQTT-повідомленнями було відкрито окреме вікно PowerShell, у якому запущено клієнт-підписник:

cd "D:\Programs\Mosquitto"

.\mosquitto_sub.exe -h localhost -t "test/topic" -v

Вікно підписника залишили відкритим, після чого в іншому вікні PowerShell запустили клієнт-видавець:

cd "D:\Programs\Mosquitto"

.\mosquitto_pub.exe -h localhost -t "test/topic" -m "MQTT works"

Клієнт-видавець опублікував тестове повідомлення в топік test/topic, а клієнт-підписник отримав його та вивів у консоль. Це підтвердило працездатність обміну повідомленнями через MQTT на локальному комп’ютері.

```
PS C:\Users\acer> cd "D:\Programs\Mosquitto"
PS D:\Programs\Mosquitto> .\mosquitto_pub.exe -h localhost -t "test/topic" -m "MQTT works"
PS D:\Programs\Mosquitto>
```

```
PS C:\Users\acer> cd "D:\Programs\Mosquitto"
PS D:\Programs\Mosquitto> .\mosquitto_sub.exe -h localhost -t "test/topic" -v
test/topic MQTT works
```

Рис.3.2-3.3 Результат виконання команд

4. Спроєктувати структуру MQTT-топіків (наприклад, object/{id}/sensor/{name}) та формат повідомлень телеметрії у JSON (значення, часова мітка, ідентифікатор сенсора, одиниці вимірювання).

4.1. Структура топіків

Використаємо ідентифікатор об’єкта scooter_01 та структуру топіків:

object/scooter_01/sensor/{sensor_id}

Цей варіант зручний, тому що кожен сигнал має окремий топік, а його ідентифікатор збігається з реєстром сигналів. У майбутньому можна буде додати інші електроскутери, змінивши лише ідентифікатор об’єкта.

№	Сигнал	Ідентифікатор сигналу	MQTT-топік
1	Швидкість руху	SIG_VEL_01	object/scooter_01/sensor/SIG_VEL_01
2	Частота обертання електродвигуна	SIG_RPM_02	object/scooter_01/sensor/SIG_RPM_02
3	Струм акумуляторної батареї	SIG_CUR_03	object/scooter_01/sensor/SIG_CUR_03
4	Напруга акумуляторної батареї	SIG_VOL_04	object/scooter_01/sensor/SIG_VOL_04
5	Температура електродвигуна	SIG_TMP_05	object/scooter_01/sensor/SIG_TMP_05
6	Рівень заряду акумуляторної батареї	SIG_SOC_06	object/scooter_01/sensor/SIG_SOC_06
7	Стан гальмування	SIG_BRK_07	object/scooter_01/sensor/SIG_BRK_07

8	Стан протиугінного блокування	SIG_LCK_08	object/scooter_01/sensor/SIG_LCK_08

Для приймання всіх повідомлень цього електроскутера підписник зможе використовувати шаблон:

object/scooter_01/sensor/#

Символ # означає, що підписник отримуватиме повідомлення з усіх вкладених топіків після object/scooter_01/sensor/. Якщо знадобиться отримувати дані лише одного сенсора, можна підписатися на конкретний топік.

4.2. Формат JSON-повідомлення

Для передавання телеметрії пропонується використовувати формат JSON. Кожне повідомлення міститиме ідентифікатор сенсора, назву сигналу, виміряне значення, одиницю вимірювання та часову мітку.

Наприклад, повідомлення про швидкість електроскутера може мати такий вигляд:

```
{
  "sensor_id": "SIG_VEL_01",
  "name": "Швидкість",
  "value": 32.5,
  "unit": "км/год",
  "timestamp": "2026-10-18T10:30:15.200000+00:00"
}
```

У цьому прикладі sensor_id визначає сигнал відповідно до реєстру, name містить його назву, value – поточне значення, unit – одиницю вимірювання, a timestamp – час формування повідомлення в часовому поясі UTC.

Для дискретних сигналів value міститиме логічне значення true або false. Наприклад, повідомлення про натискання гальма:

```
{
  "sensor_id": "SIG_BRK_07",
  "name": "Стан гальмування",
```

```
  "value": true,
  "unit": "bool",
  "timestamp": "2026-10-18T10:30:15.200000+00:00"
}
```

Реалізація формування JSON-повідомлень в емуляторі

Визначена структура JSON-повідомлень уже реалізована в програмному емуляторі сенсорів, розробленому на попередньому етапі. Формування повідомлень здійснюється методом make_message(), який створює словник із п’ятьма полями: ідентифікатором сенсора (sensor_id), назвою параметра (name), його значенням (value), одиницею вимірювання (unit) і часовою міткою (timestamp).

Фрагмент реалізації методу:

```
def make_message(self, sensor_id, name, value, unit):
    return {
        "sensor_id": sensor_id,
        "name": name,
        "value": value,
        "unit": unit,
        "timestamp": self.timestamp(),
    }
```

Метод timestamp() забезпечує формування часової мітки у форматі ISO 8601 із зазначенням часового поясу UTC.

Для формування повідомлень із різними частотами оновлення використовується метод generate_group(). Він повертає список повідомлень для відповідної групи сенсорів: 10Hz – швидкість руху та частота обертання двигуна, 1Hz – струм і напруга акумуляторної батареї, 0.1Hz – температура двигуна та рівень заряду батареї. Значення фізичних величин округлюються до визначеної кількості знаків після коми, а для окремих параметрів додатково обмежуються допустимими діапазонами.

Наприклад, для групи сенсорів із частотою 10 Гц метод формує повідомлення про швидкість та оберти двигуна:

```
if group == "10Hz":
    return [
        self.make_message(
```

```
            "SIG_VEL_01", "Швидкість",
            round(speed, 2), "км/год"
        ),
        self.make_message(
            "SIG_RPM_02", "Оберти двигуна",
            round(rpm, 1), "об/хв"
        ),
    ]
```

Дискретні сигнали обробляються методом generate_event_messages(). Він порівнює поточні стани гальмування та протиугінного блокування з попередніми значеннями. Повідомлення формується лише в разі зміни відповідного стану. Наприклад, під час зміни стану гальмування створюється повідомлення з ідентифікатором SIG_BRK_07, логічним значенням True або False та одиницею типу bool. Аналогічно формується повідомлення для сигналу протиугінного блокування SIG_LCK_08.

У функції main() сформовані повідомлення перетворюються на JSON-рядки за допомогою json.dumps() і виводяться в консоль:

```
for message in messages:
    print(json.dumps(message, ensure_ascii=False))
```

Параметр ensure_ascii=False дає змогу зберігати українські назви параметрів і символи одиниць вимірювання без перетворення їх на послідовності Unicode-екранування.

Таким чином, у програмному емуляторі реалізовано формування телеметричних повідомлень відповідно до визначеної структури JSON, зокрема для фізичних величин із різними частотами оновлення та дискретних подій. На цьому етапі повідомлення виводяться в консоль, а їх передавання через спроєктовані MQTT-топіки буде реалізовано на наступному етапі роботи.

5. Реалізувати компонент-видавець (publisher), який передає телеметрію від емулятора сенсорів до брокера із заданою частотою.

Для взаємодії з брокером через протокол MQTT використано бібліотеку paho-mqtt, яку встановлено за допомогою команди:

```
python -m pip install paho-mqtt
```

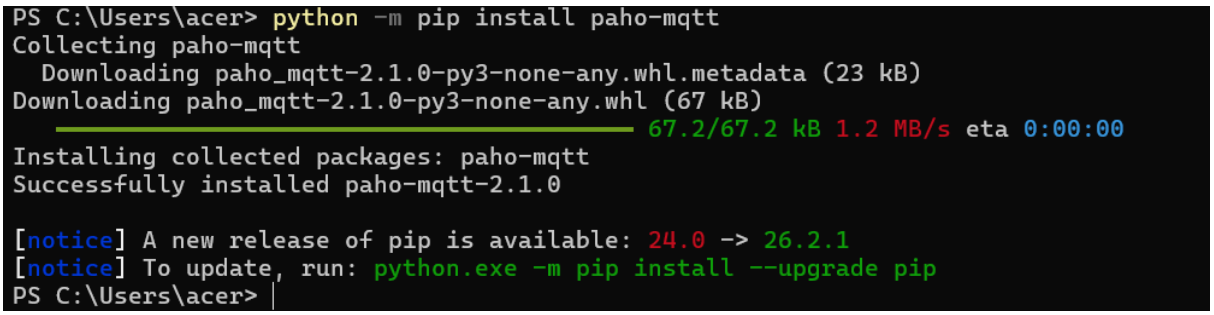


Рис. 5.1 Результат виконання команди

Для передавання телеметрії від емулятора сенсорів електроскутера до MQTT-брокера було розроблено програмний компонент emulator_publisher.py (https://drive.google.com/file/d/1ycjgEVq09lsVaNUDr2lEZ_T_UH4StB44/view?usp=sharing). Компонент використовує клас ScooterSensorEmulator із модуля emulator.py для отримання телеметричних даних.

У компоненті задано параметри підключення до локального MQTT-брокера: адресу localhost, порт 1883 та ідентифікатор клієнта scooter_publisher. Для початкового тестування передавання повідомлень використовується рівень якості обслуговування QoS 0, за якого повідомлення передаються без підтвердження їх отримання одержувачем.

Після встановлення з'єднання з брокером видавець запускає цикл передавання телеметрії. На кожному такті з інтервалом 0,1 с оновлюється модель електроскутера. Дані про швидкість руху та оберти двигуна формуються з частотою 10 Гц, дані про струм і напругу батареї – 1 Гц, а дані про температуру двигуна та рівень заряду батареї – 0,1 Гц. Повідомлення про стани гальмування та протиугінного блокування формуються у разі зміни відповідного стану.

Сформовані телеметричні дані перетворюються на повідомлення у форматі JSON і публікуються в MQTT-топіках відповідно до ідентифікаторів сенсорів. Для передавання використовується структура топіків object/scooter_01/sensor/{sensor_id}, де {sensor_id} відповідає ідентифікатору конкретного сенсора.

Для перевірки роботи компонента було організовано послідовний запуск MQTT-брокера, клієнта-підписника та видавця. Спочатку в першому вікні PowerShell запускається брокер Mosquitto в режимі докладного виведення інформації про його роботу:

cd "D:\Programs\Mosquitto"

.\mosquitto.exe -v

Далі в іншому вікні PowerShell запускається розроблений компонент-видавець:

cd "D:\Пари\Курс 5\семестр 1\двійники"

python emulator_publisher.py

Після запуску видавець підключається до брокера та починає публікувати телеметричні повідомлення. У його консолі відображаються топіки й дані повідомлень у форматі JSON.



Рис. 5.2. Результат запуску компонента-видавця emulator_publisher.py

Для додаткової перевірки передавання телеметрії можна використати консольну утиліту mosquitto_sub.exe, яка підписується на всі топіки сенсорів електроскутера:

cd "D:\Programs\Mosquitto"

.\mosquitto_sub.exe -h localhost -t "object/scooter_01/sensor/#" -v

Параметр -h localhost задає адресу брокера, -t визначає топік підписки, а -v забезпечує відображення назви топіка разом із вмістом повідомлення.



Рис. 5.3. Результат отримання телеметрії MQTT-підписником

Під час перевірки було виявлено некоректне відображення українських символів у консольному виведенні `mosquitto_sub.exe`. Спроба змінити кодову сторінку консолі на UTF-8 за допомогою команди `chcp 65001` не усунула цю проблему. Водночас отримані повідомлення підтверджують, що підписник приймає телеметричні дані, хоча українські назви датчиків і одиниці вимірювання відображаються некоректно.

Згідно з вимогами комп'ютерного практикуму, на наступному етапі передбачено розроблення власного програмного компонента-підписника `subscriber.py`, який прийматиме MQTT-повідомлення, перевірятиме

структуру JSON-даних і записуватиме отримані дані до журналу. Це дасть змогу коректно відображати текстові поля та виконувати додаткову перевірку повідомлень.

Таким чином, розроблений компонент-видавець забезпечує передавання телеметричних даних від емулятора сенсорів електроскутера до MQTT-брокера з визначеними частотами формування повідомлень. Отримання повідомлень підписником дає змогу перевірити працездатність каналу передавання телеметрії.

6. Реалізувати компонент-підписник (subscriber), який приймає повідомлення, виконує валідацію їх структури та веде журнал отриманих даних.

Для приймання та оброблення телеметричних даних, переданих через MQTT-брокер, розроблено програмний компонент subscriber.py (https://drive.google.com/file/d/1q6WnSUlvbIJCwQRVIKxM1laOqg56tHNg/view?usp=sharing) мовою Python із використанням бібліотеки paho-mqtt.

Підписник підключається до локального MQTT-брокера за адресою localhost і порту 1883 та оформлює підписку на топік object/scooter_01/sensor/#. Це дає змогу отримувати повідомлення від усіх сенсорів електроскутера.

Валідація структури повідомлень. Для перевірки отриманих даних визначено перелік обов'язкових полів: sensor_id, name, value, unit і timestamp. Після декодування повідомлення та перетворення JSON-рядка на об'єкт Python перевіряється наявність усіх обов'язкових полів і відповідність значень очікуваним типам даних.

Фрагмент реалізації перевірки:

```
payload = message.payload.decode("utf-8")
data = json.loads(payload)

if not isinstance(data, dict):
    raise ValueError("Повідомлення має бути JSON-об'єктом")

missing_fields = REQUIRED_FIELDS - data.keys()
if missing_fields:
    raise ValueError(
        f"Відсутні поля: {', '.join(sorted(missing_fields))}"
    )
```

```
    if not isinstance(data["sensor_id"], str):
        raise ValueError("sensor_id має бути рядком")

    if not isinstance(data["value"], (int, float)) or isinstance(
        data["value"], bool
    ):
        raise ValueError("value має бути числом")
```

Приймання повідомлень. Оброблення вхідних повідомлень виконується у функції on_message(), яка викликається клієнтом MQTT після отримання даних із підписаного топіка. Після успішної валідації інформація про топік і вміст повідомлення виводиться в консоль:

```
    print(f"Топік: {message.topic}")
    print(json.dumps(data, ensure_ascii=False, indent=2))
    print("-" * 50)
```

Параметр ensure_ascii=False забезпечує відображення українських символів без перетворення їх на Unicode-послідовності.

Журналювання отриманих даних. Для збереження результатів оброблення використано модуль logging. Журнал налаштовано на запис у файл telemetry.log із кодуванням UTF-8. Коректні повідомлення записуються на рівні INFO, а помилки оброблення некоректних повідомлень фіксуються на рівні WARNING.

Фрагмент налаштування журналу та запису повідомлень:

```
    logging.basicConfig(
        filename="telemetry.log",
        level=logging.INFO,
        encoding="utf-8",
        format="%(asctime)s | %(message)s"
    )

    logging.info(
        "topic=%s | data=%s",
        message.topic,
        json.dumps(data, ensure_ascii=False)
    )
```

Для запуску підписника використовується команди:

cd "D:\Пари\Курс 5\семестр 1\двійники"

python subscriber.py

MQTT-брокер і компонент-видавець запускаються відповідно до порядку, описаного в попередньому етапі.

Після встановлення з'єднання підписник отримує телеметричні повідомлення, перевіряє їхню структуру, виводить коректні дані в консоль і записує їх до журналу telemetry.log.

```
PS D:\Пари\Курс 5\семестр 1\двійники> python subscriber.py
Підключення до MQTT-брокера...
З'єднання з MQTT-брокером встановлено.
Підписка на топік: object/scooter_01/sensor/#

Топік: object/scooter_01/sensor/SIG_VEL_01
{
  "sensor_id": "SIG_VEL_01",
  "name": "Швидкість",
  "value": 4.91,
  "unit": "км/год",
  "timestamp": "2026-10-18T13:23:35.663876+00:00"
}
--------------------------------------------------
Топік: object/scooter_01/sensor/SIG_RPM_02
{
  "sensor_id": "SIG_RPM_02",
  "name": "Оберти двигуна",
  "value": 272.0,
  "unit": "об/хв",
  "timestamp": "2026-10-18T13:23:35.663876+00:00"
}
--------------------------------------------------
```

Рис. 6.1. Результат роботи компонента-підписника subscriber.py під час отримання телеметричних повідомлень.

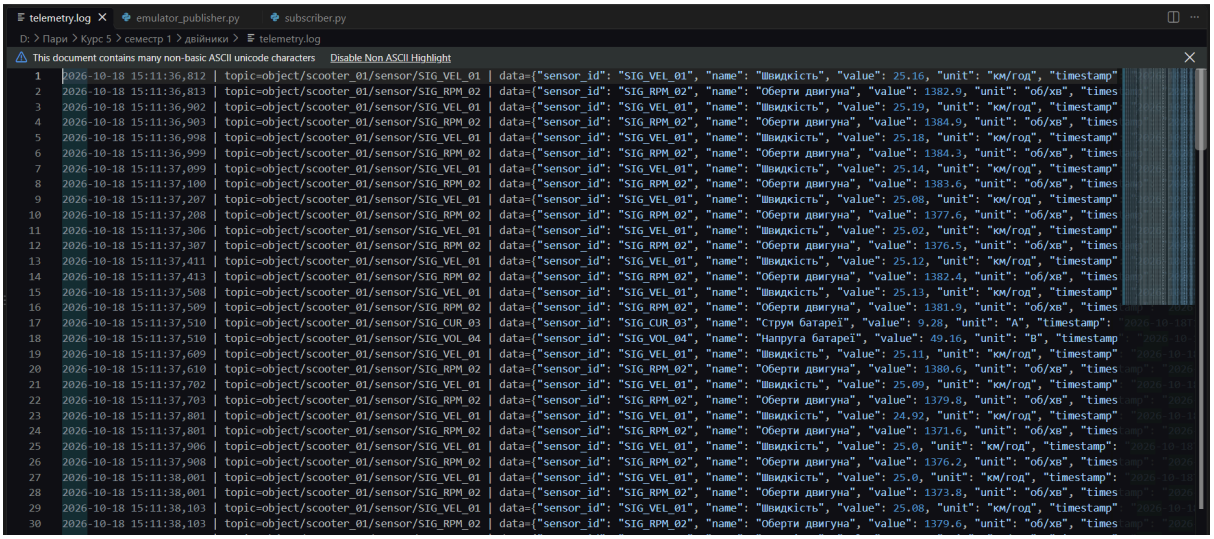


Рис. 6.2. Фрагмент журналу отриманих телеметричних даних telemetry.log.

Таким чином, розроблений компонент-підписник реалізує приймання MQTT-повідомлень, базову валідацію їхньої структури та типів даних і журналювання результатів оброблення, що відповідає вимогам шостого кроку практикуму.

7. Дослідити вплив рівнів QoS (0, 1, 2) на доставку повідомлень; налаштувати механізм Last Will для виявлення від'єднання джерела даних.

7.1 Дослідити вплив рівнів QoS (0, 1, 2) на доставку повідомлень

QoS (Quality of Service) визначає рівень гарантій доставки повідомлень у протоколі MQTT. Протокол передбачає три рівні:

- QoS 0 (At most once) – повідомлення передається без підтвердження доставки. Повторне передавання на рівні MQTT не передбачене, тому повідомлення може бути втрачено.
- QoS 1 (At least once) – передавання передбачає підтвердження отримання. У разі повторного передавання можливе отримання дублікатів повідомлення.
- QoS 2 (Exactly once) – використовується обмін службовими повідомленнями для забезпечення одноразової доставки в межах відповідного обміну MQTT.

Під час налаштування MQTT важливо розрізняти QoS публікації та QoS підписки. Публікатор задає QoS для кожного повідомлення, а підписник вказує максимальний рівень QoS, який він готовий приймати для своєї підписки. Це значення не примушує брокер підвищувати QoS повідомлення.

Фактичний QoS доставки повідомлення підписнику визначається меншим із рівня QoS опублікованого повідомлення та максимального QoS, наданого для підписки. Наприклад, якщо публікатор надсилає повідомлення з QoS 0, а підписник підписаний із QoS 2 (client.subscribe(TOPIC, qos=2)), повідомлення все одно доставляється з QoS 0.

Для дослідження впливу QoS у розробленому IoT-каналі рівень QoS публікатора змінюється послідовно між 0, 1 і 2, тоді як QoS підписки залишається незмінним і дорівнює 2. Така конфігурація дає змогу порівняти режими публікації за однакових умов підписки. Під час

експерименту оцінюються кількість переданих і отриманих повідомлень, приблизна частка втрат, затримка доставки та наявність дублікатів.

Водночас QoS не є абсолютною гарантією збереження даних у всій системі: він регламентує доставку в межах MQTT-обміну, а результат також залежить від роботи клієнтів, брокера, мережевого з'єднання та налаштувань сесії.

Для тестування впливу рівнів QoS (0, 1, 2) на доставку повідомлень було розроблено програму test_qos.py (https://drive.google.com/file/d/1uiUMRniDD3hyK6I_7mjqsbzTTXspMGpv/view?usp=sharing), яка автоматизує проведення експерименту. Тестування виконується послідовно для кожного рівня QoS протягом 30 секунд. Перед кожним запуском очищується журнал telemetry.log, після чого запускаються підписник і публікатор MQTT. Для кожного експерименту в публікаторі встановлюється відповідний рівень QoS, тоді як QoS підписки залишається незмінним і дорівнює 2.

Після завершення публікації програма надає підписнику додатковий час для оброблення отриманих повідомлень, підраховує кількість валідних повідомлень у журналі та зберігає результати у файл qos_results.json. Після проведення тестів початковий код публікатора відновлюється.

```
PS D:\Пари\Курс 5\семестр 1\двійники> python test_qos.py

============================================
ТЕСТ QoS 0
============================================
Публікація телеметрії протягом 30 с...
QoS: 0
Отримано валідних повідомлень: 658

============================================
ТЕСТ QoS 1
============================================
Публікація телеметрії протягом 30 с...
QoS: 1
Отримано валідних повідомлень: 658

============================================
ТЕСТ QoS 2
============================================
Публікація телеметрії протягом 30 с...
QoS: 2
Отримано валідних повідомлень: 658

============================================
ПІДСУМКИ
============================================
QoS 0: 658 отриманих повідомлень
QoS 1: 658 отриманих повідомлень
QoS 2: 658 отриманих повідомлень
```

Рис. 7.1. Результат виконання програми

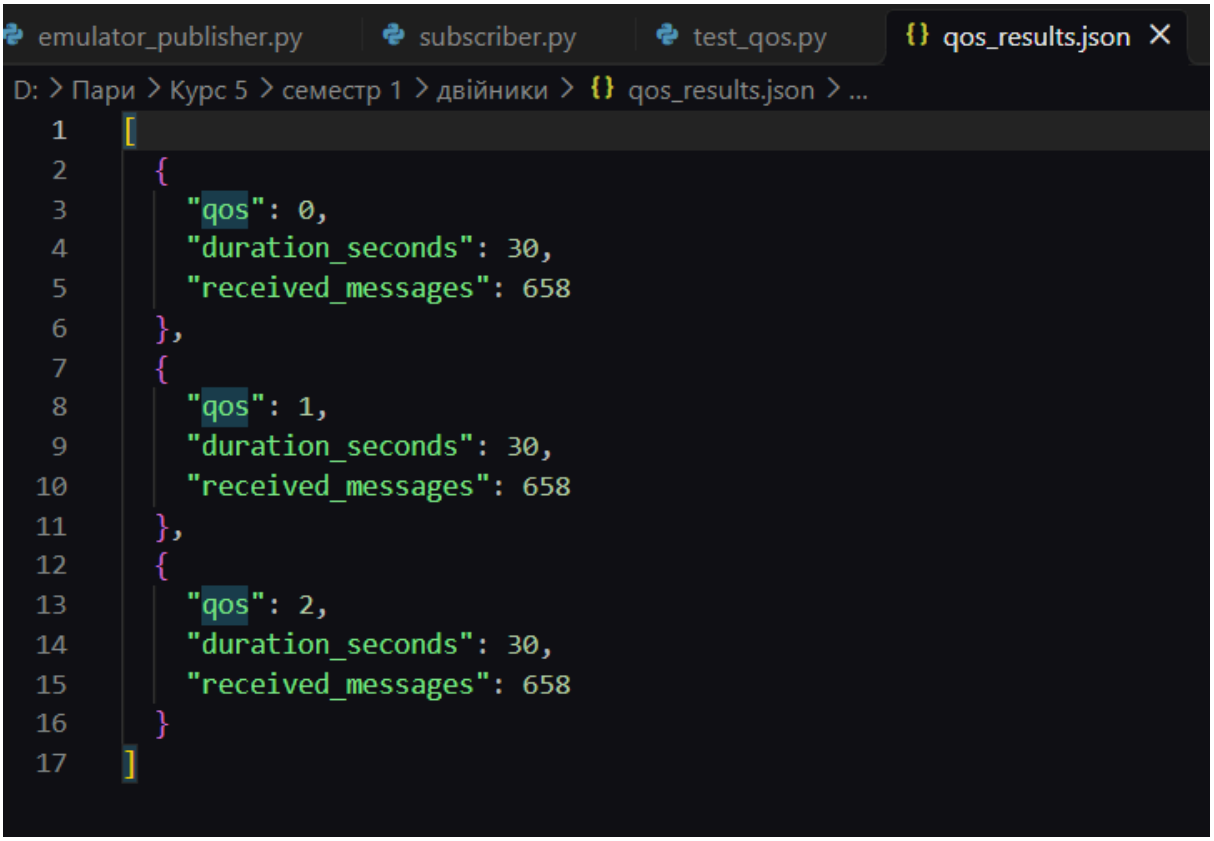


Рис. 7.2 Вміст файлу qos_results.json

У межах проведеного експерименту за тривалості публікації 30 секунд кількість отриманих валідних повідомлень була однаковою для всіх трьох рівнів QoS і становила 658 повідомлень. Це може пояснюватися тим, що тестування проводилося в локальному середовищі зі стабільним мережевим з'єднанням, за якого відмінності між рівнями QoS не проявилися в кількості отриманих повідомлень.

Для перевірки відповідності результатів заданим частотам вимірювання було розраховано очікувану кількість періодичних повідомлень за 30 секунд:

- Сигнали з частотою 10 Гц: швидкість електроскутера та частота обертання двигуна. За 30 секунд очікується $2 \times 10 \times 30 = 600$ повідомлень.
- Сигнали з частотою 1 Гц: струм і напруга акумулятора. За 30 секунд очікується $2 \times 1 \times 30 = 60$ повідомлень.
- Сигнали з частотою 0,1 Гц: температура двигуна та рівень заряду акумулятора. За 30 секунд очікується $2 \times 0,1 \times 30 = 6$ повідомлень.

Отже, очікувана кількість періодичних повідомлень становить (600 + 60 + 6 = 666). Додатково можуть формуватися подієві повідомлення про зміну станів гальм і протиугінного блокування.

Під час експерименту підписник зафіксував 658 валідних повідомлень, що на 8 повідомлень менше за розрахункову кількість періодичних повідомлень. Проте цю різницю не можна однозначно трактувати як втрату восьми повідомлень, оскільки фактична кількість сформованих повідомлень на стороні публікатора не вимірювалася, а кількість подієвих повідомлень також могла вплинути на загальний результат.

Таким чином, експеримент підтвердив працездатність MQTT-каналу та продемонстрував однакову кількість отриманих валідних повідомлень для QoS 0, QoS 1 і QoS 2 у перевірених умовах. Водночас для обґрунтованого висновку про втрати, дублювання та затримки доставки необхідно додатково підраховувати сформовані й отримані повідомлення, а також вимірювати час їх передавання.

7.2 Налаштувати механізм Last Will для виявлення від'єднання джерела даних.

Механізм Last Will and Testament (LWT) у протоколі MQTT дає змогу сповіщати підписників про нештатне від'єднання клієнта. Під час встановлення з'єднання публікатор передає брокеру параметри спеціального повідомлення: тему, вміст, рівень QoS та прапорець збереження. Якщо з'єднання переривається нештатно, брокер публікує це повідомлення після виявлення розриву.

Для впровадження цього механізму в IoT-канал електроскутера було внесено зміни до програм публікатора та підписника MQTT.

У програмі публікатора emulator_publisher.py перед встановленням з'єднання з брокером додано налаштування Last Will. Для повідомлення про нештатне від'єднання визначено тему object/scooter_01/status, вміст offline та рівень QoS 1. Також увімкнено збереження повідомлення за допомогою прапорця retain=True. Після успішного підключення публікатор надсилає повідомлення online, яке вказує на доступність джерела даних.

У програмі підписника subscriber.py додано підписку на тему object/scooter_01/status із рівнем QoS 1 та оброблення отриманих повідомлень про стан. Це дає змогу відображати в консолі повідомлення online і offline окремо від телеметричних даних електроскутера.

Для перевірки реалізації запустимо брокер, підписник та публікатор MQTT. Після встановлення з’єднання перевіряється отримання повідомлення online. Далі публікатор примусово завершує роботу без штатного від’єднання. Після виявлення розриву з’єднання брокер має опублікувати повідомлення offline, яке отримує підписник.

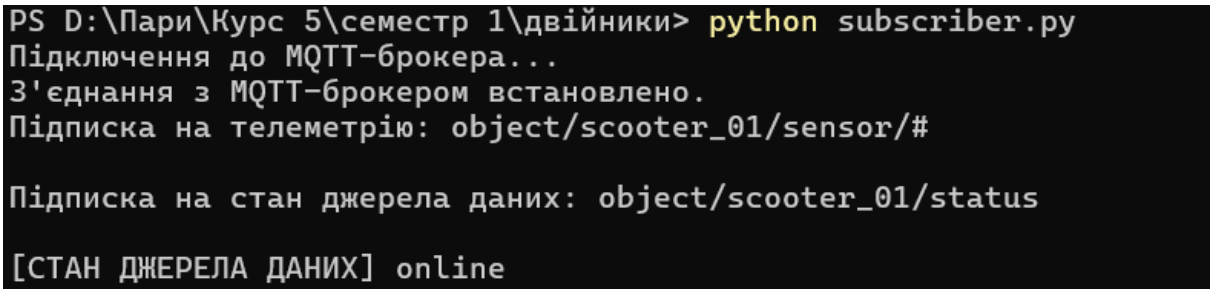


Рис. 7.3 Отримання повідомлення online після підключення публікатора MQTT.

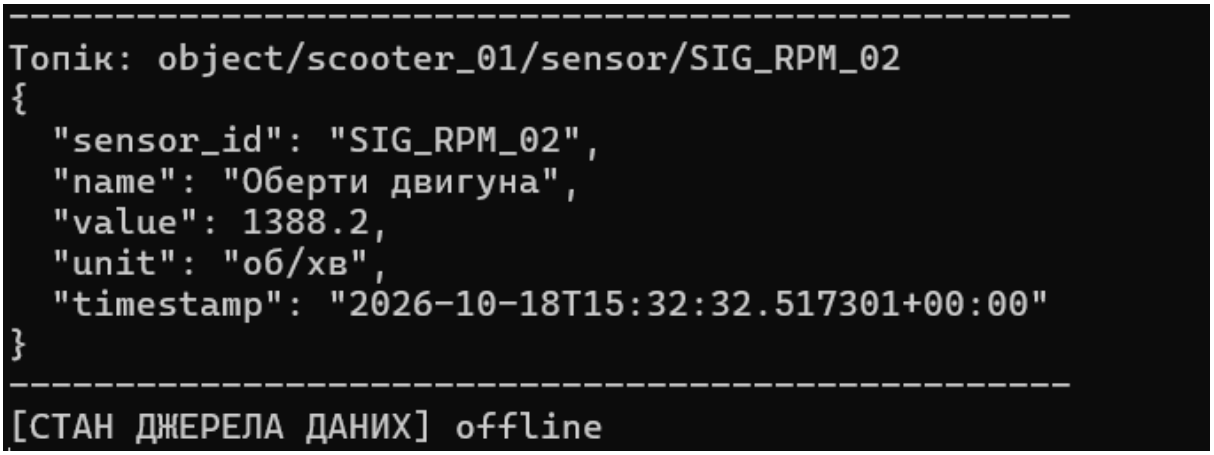


Рис. 7.4 Отримання повідомлення offline після нештатного від’єднання публікатора MQTT.

Зауважу, що термінал публікатора потрібно закрити, а не зупиняти його роботу за допомогою `Ctrl+C`, оскільки в останньому випадку спрацьовує штатне від’єднання від брокера MQTT і механізм Last Will не активується.

Таким чином, упровадження Last Will доповнює IoT-канал механізмом сповіщення про нештатне від’єднання джерела даних. Для підтвердження працездатності реалізації необхідно перевірити отримання повідомлень про обидва стани та зафіксувати результат у консолі підписника.

8. Перевірити працездатність каналу: одночасна робота кількох емульованих сенсорів, оцінка втрат і затримок повідомлень; оформити звіт.

Для перевірки працездатності розробленого IoT-каналу даних електроскутера використаємо програму channel_test.py (https://drive.google.com/file/d/1sfRqSI2FpW7uV78rkmwMzp2Q4E_bTIT6/view?usp=sharing), реалізовану мовою Python. Вона автоматизує перевірку передавання телеметрії через MQTT-брокер, підрахунок отриманих повідомлень від різних сенсорів і оцінювання затримки їх надходження.

Для оцінювання втрат використаємо порівняння очікуваної та фактичної кількості отриманих періодичних повідомлень. Очікувану кількість визначимо на основі частоти передавання даних кожного сенсора та тривалості тестування. Наприклад, сенсор зі швидкістю оновлення 10 Гц має передати приблизно 600 повідомлень за 60 секунд, а сенсор із частотою 1 Гц – приблизно 60 повідомлень.

Такий підхід дає змогу перевірити одночасну роботу кількох сенсорів без додавання до кожного повідомлення окремого порядкового номера для контролю послідовності. Він спрощує реалізацію тестування та дає змогу визначити загальну нестачу повідомлень і перевірити кількість даних від кожного сенсора.

Водночас цей метод має обмеження. Він не дає змоги точно встановити, які саме повідомлення були втрачені, виявити дублювання або відрізнити втрату повідомлень від відхилень, пов’язаних із фактичним часом запуску та передавання даних. Крім того, розрахунок залежить від правильності заданих частот і тривалості тесту. Для точнішого визначення втрат можна додатково вести лічильник сформованих повідомлень на стороні видавця та порівнювати його з кількістю отриманих повідомлень. Іншим варіантом є додавання порядкового номера до кожного повідомлення.

Перед початком тестування переконаємося, що MQTT-брокер Eclipse Mosquitto запущений. Для запуску тестування відкриємо PowerShell, перейдемо до каталогу проєкту та виконаємо команду:

```
cd "D:\Пари\Курс 5\семестр 1\двійники"
python channel_test.py
```

Після запуску програма підключиться до MQTT-брокера та оформить підписку на топік object/scooter_01/sensor/#, щоб отримувати повідомлення від усіх сенсорів електроскутера. Далі автоматично запуститься

компонент-видавець emulator_publisher.py, який формуватиме та передаватиме телеметрію з визначеними частотами оновлення.

Тестування триватиме 60 секунд від моменту отримання першого повідомлення. Під час його виконання програма перевірятиме структуру JSON-повідомлень, наявність обов'язкових полів, відповідність ідентифікаторів сенсорів MQTT-топікам і типи значень. Для коректних повідомлень також визначатиметься приблизна затримка між часовою міткою формування телеметрії та моментом її отримання.

```
PS D:\Пари\Курс 5\семестр 1\двійники> python channel_test.py
Підключення до MQTT-брокера успішне.
Підписка активна: object/scooter_01/sensor/#
Очікування першого повідомлення. Тест: 60 с.

========== РЕЗУЛЬТАТИ ТЕСТУ ==========
Тривалість тесту: 60 с
Очікувані періодичні повідомлення: 1332
Отримані періодичні повідомлення: 1332
Оцінка нестачі повідомлень: 0
Оцінка втрат: 0.000%
Некоректні повідомлення: 0
Невідомі сенсори: 0
Середня затримка: 1.084 мс
Максимальна затримка: 2.998 мс
95-й процентиль: 2.023 мс

Повідомлення за сенсорами:
SIG_VEL_01 (Швидкість): 600/600
SIG_RPM_02 (Оберти двигуна): 600/600
SIG_CUR_03 (Струм батареї): 60/60
SIG_VOL_04 (Напруга батареї): 60/60
SIG_TMP_05 (Температура двигуна): 6/6
SIG_SOC_06 (Заряд батареї): 6/6
SIG_BRK_07 (Стан гальмування): 2 повідомлень
SIG_LCK_08 (Протиугінне блокування): 4 повідомлень

Примітка: оцінка втрат приблизна; MQTT QoS 0 не підтверджує доставку.
Тест завершено.
```

Рис. 8. Результат тестування IoT-каналу передавання телеметрії електроскутера за допомогою channel_test.py.

За результатами тестування тривалістю 60 секунд було отримано 1332 періодичні повідомлення з 1332 очікуваних. Розрахована нестача становила 0 повідомлень. Це свідчить про те, що кількість отриманих періодичних повідомлень відповідає очікуваній. Некоректних повідомлень і повідомлень від невідомих сенсорів не виявлено.

Перевірка кількості повідомлень за окремими сенсорами також підтвердила відповідність очікуваним значенням. Від сенсорів швидкості

та обертів двигуна було отримано по 600 повідомлень, від сенсорів струму та напруги батареї – по 60, а від сенсорів температури двигуна та рівня заряду батареї – по 6 повідомлень. Крім того, було отримано 2 подієві повідомлення про стан гальмування та 4 повідомлення про стан протиугінного блокування. Кількість подієвих повідомлень залежить від змін стану відповідних сигналів, тому для них не розраховувалася очікувана кількість за періодичною частотою.

Середня затримка між часовою міткою формування телеметрії та її отриманням становила 1,084 мс, максимальна – 2,998 мс, а 95-й процентиль – 2,023 мс. Це означає, що в межах проведеного тесту затримки були невеликими. Водночас ці значення характеризують виміряну затримку в умовах конкретного тестування і не гарантують аналогічних результатів за іншого навантаження.

Отже, тестування підтвердило працездатність розробленого IoT-каналу та одночасне отримання телеметрії від шести періодичних сенсорів і двох подієвих сигналів. Проте оцінка втрат є приблизною: використання MQTT QoS 0 не передбачає підтвердження доставки кожного повідомлення, а збіг очікуваної та фактичної кількості не є абсолютним доказом відсутності втрат.

Висновок

У ході виконання комп’ютерного практикуму було успішно розроблено та досліджено програмні компоненти IoT-каналу даних для платформи цифрового двійника електроскутера.

Метою роботи було набуття практичних умінь інтеграції сенсорних даних у програмні системи та застосування протоколів передавання даних інтернету речей, що було повністю реалізовано шляхом створення програмного емулятора сенсорів і компонентів передавання та приймання телеметрії через брокер.

Під час виконання завдань було визначено вісім базових телеметричних сигналів із різними частотами опитування, діапазонами та одиницями вимірювання. Розроблено програмний емулятор мовою Python, який генерує реалістичні часові ряди з урахуванням фізики поїздки, добового циклу температури, випадкового вимірювального шуму та аварійних аномалій.

Для організації зв'язку розгорнуто локальний MQTT-брокер Eclipse Mosquitto, спроєктовано ієрархічну структуру топіків та уніфікований формат обміну даними у форматі JSON. Реалізовано компонент-видавець

для трансляції телеметрії з визначеними частотами та компонент-підписник із функцією валідації структури й веденням журналу.

Проведені дослідження дозволили оцінити вплив рівнів якості обслуговування QoS (0, 1, 2) за допомогою автоматизованих скриптів, налаштувати механізм аварійного сповіщення Last Will and Testament для виявлення непередбачуваного від’єднання джерела даних, а також перевірити працездатність каналу через інтеграційне тестування, яке підтвердило стабільну роботу системи з мінімальними затримками передавання пакетів.

Отримана система повністю відповідає вимогам технічного завдання, демонструє надійну взаємодію клієнтів у реальному часі та готова до практичного використання і захисту.